

Programación 1 — Unidad 4: Trabajo colaborativo

Actividad 2: Repositorio, configuración y flujo de trabajo (push/pull)

Índice

- 4.2. El repositorio y las tres áreas
- 4.3. Configuración inicial
- 4.4. El flujo de trabajo básico
- 4.5. Trabajar con un repositorio remoto

4.2. El repositorio y las tres áreas

Un **repositorio** (o «repo» para abreviar) es la carpeta de tu proyecto con seguimiento de Git. Git organiza el trabajo en tres áreas por las que va pasando cada cambio antes de quedar guardado para siempre en la nube. Entender este recorrido es la clave de Git.

Flujo de trabajo con Git y GitHub



El recorrido de un cambio: del directorio de trabajo al área de preparación, al repositorio local y finalmente al remoto.

Área	Qué contiene	Cómo se llega
Directorio de trabajo	Tus archivos tal como los editás	Editar el código
Área de preparación (staging)	Los cambios elegidos para guardar	<code>git add</code>
Repositorio local	La historia confirmada en tu PC	<code>git commit</code>
Repositorio remoto (GitHub)	La historia compartida en la nube	<code>git push</code>



Diagrama 4.2 generado con Mermaid.

4.3. Configuración inicial

La primera vez que usás Git en una computadora, le decís quién sos. Estos datos quedarán asociados a cada cambio que registres, para que se sepa quién hizo qué.

```
git config --global user.name "Martín García"
git config --global user.email "martin@ejemplo.com"
```

Qué hace el código:

- `git config` es el comando para configurar Git. La palabra `--global` significa «para todos mis proyectos».
- `user.name` guarda tu nombre, que aparecerá en cada cambio que registres.
- `user.email` guarda tu correo (conviene usar el mismo con el que te registrás en GitHub).
- Esto se hace una sola vez por computadora; después Git ya sabe quién sos.

4.4. El flujo de trabajo básico

El ciclo de trabajo diario con Git se reduce a unos pocos comandos. Supongamos que tenés una carpeta con tu proyecto y querés empezar a versionarlo:

```
git init
git status
git add archivo.py
git commit -m "Mensaje descriptivo del cambio"
git push
```

Qué hace el código:

- `git init` crea el repositorio: le dice a Git que empiece a seguir los cambios de esa carpeta. Se hace una sola vez.
- `git status` muestra el estado actual: qué archivos cambiaron y cuáles están listos para guardar.
- `git add archivo.py` prepara ese archivo para guardarlo (lo lleva al área de staging). Con `git add .` preparás todos.
- `git commit -m "..."` confirma los cambios preparados, dejándolos en el historial con un mensaje que los describe.
- `git push` envía esos commits al repositorio remoto en GitHub, para compartirlos o respaldarlos.

Buenas prácticas

Escribí mensajes de commit claros y en presente: «Agrega validación de edad» se entiende mucho mejor que «cambios» o «arreglos varios». Tu yo del futuro, y tus compañeros, te lo van a agradecer cuando busquen algo en el historial.

4.5. Trabajar con un repositorio remoto

Para colaborar o respaldar tu trabajo, conectás tu repositorio local con uno remoto en GitHub. Los comandos clave son `clone` (copiar a tu PC un repo que ya existe) y `pull` (traer los cambios que otros subieron).

```
git clone https://github.com/usuario/proyecto.git
git pull
git push
```

Qué hace el código:

- `git clone <dirección>` descarga una copia completa de un repositorio remoto a tu computadora.
- `git pull` trae e incorpora los últimos cambios que otras personas subieron al remoto. Conviene hacerlo antes de empezar a trabajar.
- `git push` hace lo contrario: sube tus propios commits al remoto para que los demás los reciban.

Nota

Cuando varias personas modifican las mismas líneas, puede aparecer un **conflicto de fusión** (merge conflict). Git marca la zona en disputa para que una persona decida qué versión conservar. No es un error grave: es parte normal del trabajo en equipo.